



Università degli Studi di Padova
Corso di Laurea in Informatica
A. A. 2025/26



Relazione

Progetto di Tecnologie Web

Studenti	Sofia De Blasi (referente) - 2111014 - sofia.deblasi@studenti.unipd.it Nicola Simionato - 2113190 - nicola.simionato.5@studenti.unipd.it Nenad Radulovic - 2101059 - nenad.radulovic@studenti.unipd.it Giacomo Speggiorin - 2116438 - giacomo.speggiorin@studenti.unipd.it
Indirizzo del sito	https://tecweb.studenti.math.unipd.it/sdeblasi/
Utente	Username: user, Password: user

Indice

1	Introduzione	1
1.1	Com'è nata l'idea	1
2	Analisi del lavoro svolto	1
2.1	Processo decisionale	1
2.2	Modalità di lavoro	1
2.3	Divisione del lavoro	1
2.4	Docker	2
3	Progettazione	2
3.1	Analisi dei contenuti	2
3.2	Descrizione delle funzionalità	2
3.3	Architettura del sistema	3
3.4	Struttura del sito	3
3.5	Schema di navigazione e metafora della pesca	3
3.6	Pagine di errore	4
3.7	Database	4
4	Sviluppo	5
4.1	HTML	5
4.1.1	Meta tag <code>viewport</code>	5
4.1.2	<code><aside></code> in <code>esplora.html</code>	6
4.1.3	Struttura di <code>cardTemplate</code>	6
4.1.4	Gestione delle immagini	6
4.1.5	Form e validazione	6
4.1.6	Messaggio informativo nei redirect verso la pagina di accesso	7
4.2	CSS e decisioni di design	7
4.3	PHP	7
5	Accessibilità	8
5.1	Strumenti di verifica	8
5.2	Tipografia e leggibilità	9
5.3	Colori, contrasti e accessibilità mobile	9
6	Validazione, sicurezza e gestione degli errori	10
6.1	Errori lato client (JavaScript)	10
6.2	Errori lato server (PHP)	11
6.3	Sicurezza delle password	12
7	Configurazione <code>.htaccess</code>	12
8	SEO e ottimizzazione per i motori di ricerca	13
8.1	Analisi delle possibili ricerche degli utenti	13
8.2	Meta tag <code>title</code> , <code>description</code> e <code>keywords</code>	13
8.3	Struttura semantica e accessibilità come fattori SEO	13
8.4	Performance	14
9	Test	14

9.1	Testing multiplatforma con e senza JavaScript	14
9.2	Ordine di tabulazione e navigazione da tastiera	14
9.3	Analisi delle prestazioni e SEO	15
9.4	Validazione con strumenti automatici	15
9.5	Studio comportamento utenti	15

10	Guida all'installazione del progetto	15
-----------	---	-----------

1 Introduzione

1.1 Com'è nata l'idea

L'idea del progetto nasce dall'esigenza, comune tra gli studenti universitari, di individuare rapidamente eventi, attività e servizi utili nella propria città universitaria. Attualmente tali informazioni risultano frammentate tra diversi canali (pagine social, bacheche fisiche sparse, etc.), rendendo la ricerca poco immediata.

Da questa osservazione è emersa la necessità di una piattaforma unica che centralizzi la pubblicazione di eventi e iniziative, permetta agli studenti di consultarli senza monitorare più fonti e offra agli organizzatori un punto di raccolta comune.

La riflessione si è estesa anche ad altri servizi rilevanti per la vita universitaria come ripetizioni, affitti e progetti sperimentali. Da qui l'idea di creare uno spazio organizzato, accessibile e pensato per raccogliere tutto ciò che può risultare utile nella quotidianità studentesca.

2 Analisi del lavoro svolto

2.1 Processo decisionale

Durante lo sviluppo del progetto sono state necessarie numerose decisioni riguardanti design, funzionalità e contenuti. La maggior parte delle scelte è stata discussa in presenza tra tutti i membri del gruppo, ma sono stati adottati anche canali sincroni e asincroni.

Per le comunicazioni asincrone è stata utilizzata la piattaforma di messaggistica **WhatsApp**, mentre **Discord** è stato impiegato come supporto alle riunioni e per la gestione delle comunicazioni sincrone a distanza.

2.2 Modalità di lavoro

Per garantire flessibilità organizzativa, il gruppo ha scelto una modalità di lavoro prevalentemente asincrona.

Le attività sono state distribuite in modo che ciascun componente lavorasse su aspetti differenti, evitando sovrapposizioni. A supporto di questa organizzazione è stato adottato un sistema di versionamento tramite **Git** con hosting su **GitHub**, utile per tracciare modifiche, gestire branch e mantenere un flusso di lavoro ordinato.

2.3 Divisione del lavoro

Considerata la natura accademica del progetto, si è scelto di coinvolgere ogni membro del gruppo in tutte le principali aree di sviluppo, così da garantire una comprensione completa del funzionamento della piattaforma e una partecipazione equilibrata.

La progettazione e la realizzazione del sito sono state quindi affrontate in modo collaborativo: tutti i componenti hanno contribuito alla struttura HTML, allo sviluppo dei fogli di stile, all'implementazione della logica PHP, alla progettazione del database, ai test di accessibilità e alla validazione lato client e lato server.

Per coordinare il lavoro sono state organizzate riunioni periodiche di allineamento e sono state utilizzate le **GitHub Issues**, che hanno permesso di tracciare in modo chiaro le attività, le priorità e lo stato di avanzamento, garantendo una distribuzione bilanciata dei compiti.

2.4 Docker

Per garantire un ambiente di sviluppo uniforme tra i membri del gruppo è stato utilizzato Docker. È stata predisposta un'immagine contenente:

- un database **MariaDB** (v. 10.6.7);
- un server **Apache PHP** (v. 8.2);
- un'istanza di **phpMyAdmin**.

Questa configurazione ha permesso di evitare differenze tra sistemi operativi, velocizzare il lavoro e avviare tutti i servizi necessari tramite un unico **docker-compose**.

3 Progettazione

3.1 Analisi dei contenuti

La piattaforma funge da bacheca universitaria online, articolata in quattro categorie principali: **Affitti**, **Esperimenti**, **Eventi** e **Ripetizioni**. Ogni annuncio condivide una struttura comune, arricchita da attributi specifici in base alla categoria.

Campi generici Ogni annuncio contiene:

- **Categoria** (una delle quattro disponibili);
- **Titolo e Descrizione**;
- **Data di pubblicazione** (generata automaticamente);
- **Città** di riferimento;
- **Autore** (utente registrato).

Campi specifici Ogni categoria prevede tre attributi dedicati:

- | | |
|---|---|
| • Affitti : prezzo mensile, indirizzo, numero coinquilini. | • Eventi : data, luogo, costo entrata. |
| • Esperimenti : laboratorio, durata prevista, compenso. | • Ripetizioni : materia, livello, prezzo orario. |

3.2 Descrizione delle funzionalità

Registrazione e accesso Gli utenti possono registrarsi tramite un form dedicato e accedere alle funzionalità riservate.

Pubblicazione e gestione annunci Gli utenti autenticati possono creare, modificare ed eliminare i propri annunci. Gli annunci eliminati vengono rimossi anche dalle liste dei preferiti.

Ricerca e filtraggio Il sistema offre: **filtri generici** (categoria, data), **filtri specifici** per categoria e **ricerca testuale** su titolo, descrizione ed email dell'autore.

Validazione Nei form con attributo `data-validate`, l'invio viene bloccato in caso di dati non validi. La validazione è gestita sia lato client sia lato server (si veda [la sezione sulla validazione](#)).

3.3 Architettura del sistema

Il progetto segue una chiara separazione tra: **struttura** (HTML), **presentazione** (CSS), **comportamento** (JavaScript), **logica applicativa** (PHP) e **persistenza** (database).

La composizione delle pagine avviene tramite template HTML presenti in `/pages`, nei quali PHP inserisce dinamicamente i contenuti. Le risorse statiche sono organizzate in cartelle dedicate (`/css`, `/js`, `/assets`, `/img_annunci`).

3.4 Struttura del sito

La struttura del sito è stata analizzata tramite un modello ad albero (vedi Figura 1), utile per valutarne ampiezza e profondità ed evidenziare i percorsi di navigazione principali.

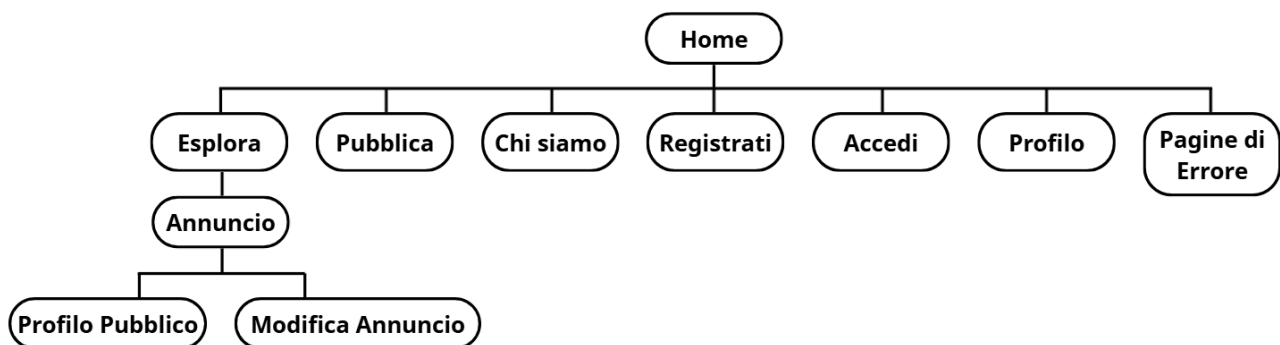


Figura 1: Rappresentazione ad albero della struttura del sito

Dal punto di vista dell'architettura dell'informazione, il sito presenta una **ampiezza (7) maggiore rispetto alla profondità (4)**: le funzionalità principali sono raggiungibili direttamente dalla homepage o con al massimo tre livelli di navigazione. Questa scelta riduce il carico cognitivo e facilita l'orientamento, soprattutto per utenti alla prima visita.

3.5 Schema di navigazione e metafora della pesca

La navigazione del sito combina struttura gerarchica e percorsi trasversali, richiamando la **metafora della pesca** per rappresentare i diversi modi in cui l'utente può ricercare i contenuti:

- **Barra di ricerca** → *tiro perfetto*: permette di individuare rapidamente un annuncio specifico.

- **Filtri generici e specifici** → *trappola per aragoste*: restringono progressivamente il campo dei risultati.
- **Accesso diretto alle categorie** → *pesca con la rete*: consente di esplorare insiemi omogenei di contenuti.
- **Ultimi annunci inseriti** → *pesca con la rete*: favorisce la scoperta casuale.
- **Preferiti e area personale** → *boa di segnalazione*: permette di raccogliere e ritrovare facilmente i contenuti di interesse.

3.6 Pagine di errore

Ogni errore è stato progettato per non essere bloccante: sono sempre presenti elementi di interazione chiari, come il pulsante per tornare alla *Home* o il collegamento diretto alla bacheca degli annunci. Questo permette all'utente di recuperare rapidamente la navigazione senza sentirsi "intrappolato" nella pagina di errore. Inoltre, abbiamo scelto di utilizzare un tono leggero e leggermente ironico. La gestione degli errori viene effettuata sia tramite codice PHP restituendo la pagina personalizzata relativa all'errore specifico che attraverso il file `.htaccess` che reindirizza gli errori a pagine personalizzate.

3.7 Database

Il database è stato progettato per essere semplice, normalizzato e facilmente estendibile, in linea con le esigenze di una bacheca universitaria. Le entità principali sono: **Utente**, **Città**, **Annuncio**, le quattro specializzazioni di **Annuncio**, **ImmaginiAnnuncio** e **Preferiti**.

Schema ER La figura (vedi Figura 2) mostra il modello concettuale, in cui **Annuncio** rappresenta l'entità centrale e viene estesa da quattro specializzazioni, una per ciascuna categoria (**Affitti**, **Esperimenti**, **Eventi**, **Ripetizioni**). Ogni specializzazione aggiunge tre attributi specifici, evitando duplicazioni nei campi generici.

Ristrutturazione della generalizzazione In fase logica, la generalizzazione è stata trasformata in quattro relazioni IS-[categoria], ciascuna collegata a **Annuncio** tramite chiave primaria condivisa (vedi Figura 3). Questa scelta mantiene la normalizzazione e semplifica l'estendibilità del modello.

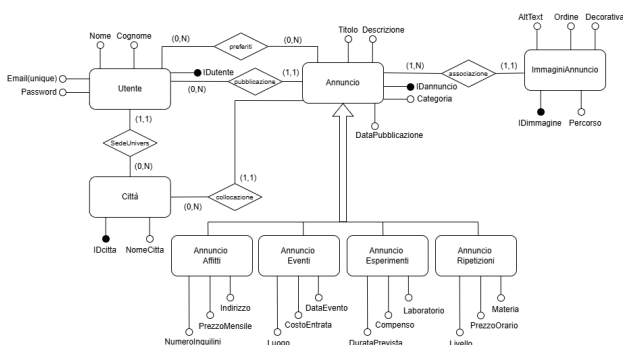


Figura 2: Schema ER

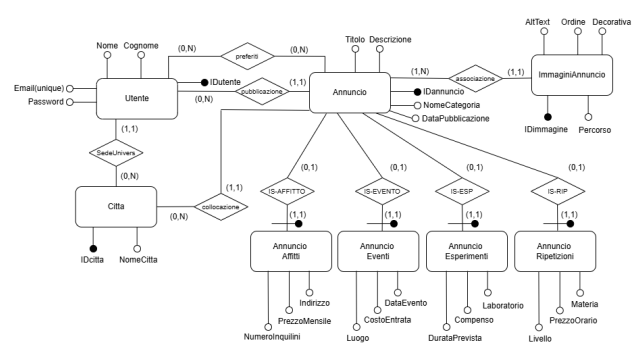


Figura 3: Schema ER ristrutturato

Schema relazionale Le tabelle principali sono:

- **Utente**(IdUtente, Nome, Cognome, Email [UNIQUE], Password, IdCittà*)
- **Città**(Idcittà, Nomecittà [UNIQUE])
- **Annuncio**(IdAnnuncio, Titolo, Descrizione, DataPubblicazione, Idutente, Categoria, IdCittà)
- **Affitti**(IdAnnuncio, PrezzoMensile, Indirizzo, NumeroInquilini)
- **Esperimenti**(IdAnnuncio, Laboratorio, DurataPrevists, Compenso)
- **Eventi**(IdAnnuncio, DataEvento, Luogo, CostoEntrata)
- **Ripetizioni**(IdAnnuncio, Materia, Livello, PrezzoOrario)
- **ImmaginiAnnuncio**(IdImmagine, IdAnnuncio, Percorso, AltText*, Decorativa, Ordine)
- **Preferiti**(IdAnnuncio, IdUtente)[UNIQUE]

Le relazioni sono collegate tramite chiavi esterne con vincoli **ON DELETE CASCADE** nelle tabelle dipendenti (specializzazioni, immagini, preferiti), così da mantenere la coerenza dei dati in caso di eliminazione di un annuncio.

Normalizzazione Lo schema rispetta la 3NF:

- attributi atomici (1NF);
- dipendenza completa dalla chiave primaria nelle specializzazioni (2NF);
- assenza di dipendenze transitive (3NF).

Scelte progettuali Sono stati adottati ID numerici auto-incrementali per garantire stabilità, efficienza nelle join e uniformità dello schema. La separazione tra **Annuncio** e le sue specializzazioni consente di aggiungere nuove categorie senza modificare la struttura esistente.

4 Sviluppo

4.1 HTML

Ogni documento è organizzato nelle sezioni `<head>` e `<body>`, con uso coerente di elementi semantici ed evitando tag puramente presentazionali.

4.1.1 Meta tag viewport

È stato inserito il meta tag: `<meta name="viewport" content="width=device-width"/>` che consente ai CSS di riferirsi alla larghezza effettiva del dispositivo, evitando l'applicazione del layout desktop su schermi ad alta densità di pixel.

4.1.2 <aside> in esplora.html

Nella pagina `esplora.html` l'elemento `<aside>` contiene i filtri di ricerca, poiché rappresentano contenuti accessori rispetto al flusso principale (lista degli annunci). Essendo una funzionalità secondaria all'interno della pagina sono stati adottati alcuni compromessi con JavaScript per accessibilità e usabilità. In particolare, con la disattivazione di JavaScript, da telefono non è possibile aprire i filtri, che diventano una tendina laterale, su desktop invece rimanendo sempre visibili non possono essere raggiunti tramite il tab, questo a causa dell'uso dell'attributo `inert`.

4.1.3 Struttura di cardTemplate

Le card degli annunci (`cardTemplate.html`) utilizzano `<h3>` per i titoli, evitando coppie `<dt>`–`<dd>`. La gerarchia semantica è così preservata: `<h2>` identifica la sezione (es. “Ultimi inseriti”), mentre `<h3>` rappresenta ciascun annuncio.

4.1.4 Gestione delle immagini

La gestione delle immagini è progettata per ottimizzare prestazioni e accessibilità.

- Le immagini above-the-fold usano `loading="eager"`, tutte le altre `loading="lazy"`.
- Formati adottati: WebP per immagini decorative, SVG per il logo e JPG per le immagini degli annunci.
- Tutte le immagini sono inferiori a 1 MB.

Le immagini decorative sono inserite nel markup e dimensionate via CSS; il logo utilizza tecniche di image replacement per mantenere accessibile il testo alternativo, mentre le immagini degli annunci hanno dimensioni fissate via HTML per rendere l'esperienza utente migliore in quanto ciò garantisce un rendering più veloce.

Nel form di pubblicazione/modifica l'utente deve rispettare vincoli espliciti (max 1 MB, formato JPG), infatti è presente un controllo sulla dimensione dei file.

Si è scelto di limitare i MIME type accettati, rinunciando a conversioni automatiche o all'utilizzo del tag `<picture>` con doppia estensione `.png` per la stampa e `.jpg` per il web, per privilegiare semplicità e controllo.

La gestione dell'attributo `alt` è affidata all'utente: ogni immagine deve avere un testo alternativo oppure essere marcata come decorativa, e non è possibile lasciare entrambi i campi vuoti. Il testo alternativo è limitato a 100 caratteri per garantire sintesi e usabilità con screen reader.

4.1.5 Form e validazione

La struttura dei form utilizza `fieldset` e `legend`, anche quando destinati principalmente alle tecnologie assistive, questo fa sì che nessun campo sia figlio diretto di `form`.

Ogni input utilizza il tipo HTML più appropriato ed è sempre associato a una `label`. Dove utile, sono abilitati `autocomplete` e `spellcheck` adeguati.

La validazione sfrutta: controlli HTML5 (`required`, `min`, `max`, `step`, vincoli di tipo), validazione JavaScript per feedback più dettagliati ed istantanei e validazione lato server per garantire correttezza e sicurezza.

4.1.6 Messaggio informativo nei redirect verso la pagina di accesso

Quando l'utente viene reindirizzato alla pagina di accesso a seguito del tentativo di accedere a una funzionalità riservata, viene mostrato automaticamente un riquadro informativo che spiega il motivo del redirect, con un messaggio contestuale alla pagina di provenienza (pubblicazione, modifica, profilo o salvataggio nei preferiti).

4.2 CSS e decisioni di design

I fogli di stile sono stati progettati privilegiando l'uso di dimensioni relative, così da garantire un'interfaccia adattabile a dispositivi e contesti differenti.

Per assicurare uniformità grafica è stata definita una `:root` contenente variabili CSS per colori, spaziature, raggi di arrotondamento, ombre, transizioni e una scala tipografica coerente per dimensioni e pesi dei caratteri.

Il layout è realizzato principalmente tramite Flexbox, utilizzato per strutture fluide e adattive, mentre CSS Grid è impiegato solo nei casi in cui risulta più appropriato (ad esempio per elenchi di card o sezioni a griglia).

La responsività è gestita tramite la separazione tra `style.css` e `mini.css`, con un breakpoint principale a 800px per l'adattamento ai dispositivi mobili. Sono presenti due ulteriori media query, tra cui una dedicata alla navigazione con menù hamburger, attivata sotto i 1000px per migliorare l'usabilità su tablet e schermi intermedi.

È stato inoltre previsto un foglio di stile dedicato alla stampa `print.css`, che mostra esclusivamente le informazioni essenziali, rimuovendo elementi decorativi e interattivi per ridurre il consumo di inchiostro ma mantenendo una struttura coerente del contenuto per evitare il fenomeno del disorientamento.

Particolare attenzione è stata riservata all'accessibilità visiva e all'interazione da tastiera, con la gestione esplicita degli stati di focus visibile, l'uso coerente dei contrasti e l'eliminazione di animazioni superflue nei contesti in cui potrebbero risultare problematiche, come nelle versioni mobile o di stampa.

4.3 PHP

Per consentire al sistema di generare pagine web il cui contenuto non sia statico, ma vari in base al contesto, tutte le pagine vengono composte dinamicamente tramite PHP. All'interno della cartella `/pages` sono presenti i template HTML, nei quali sono stati inseriti marcatori racchiusi tra parentesi quadrate (ad esempio `[TitoloAnnuncio]`) che vengono sostituiti a runtime con i contenuti dinamici appropriati.

Oltre alla generazione delle pagine, il PHP si occupa di:

- gestire tutte le richieste provenienti dal client tramite form;

- validare gli input dell'utente (si veda [la sezione sulla validazione](#));
- gestire le sessioni di accesso dell'utente;
- reindirizzare l'utente non autenticato verso la pagina di accesso qualora tenti di raggiungere funzionalità riservate;
- reindirizzare la navigazione verso le pagine di errore in caso di condizioni inattese;
- gestire le connessioni al database, garantendo che non rimangano aperte più del necessario, e costruire correttamente le query. A tale scopo, tutte le operazioni relative al database sono state centralizzate nel file **dbConnect.php**, così da mantenere una chiara separazione del codice.

Per semplificare lo sviluppo e aumentare la riutilizzabilità del codice, è stata inoltre realizzata una libreria interna al progetto, contenuta nel file **tool.php**.

5 Accessibilità

5.1 Strumenti di verifica

Sono stati utilizzati diversi strumenti di verifica:

- **NVDA**: per testare la gerarchia dei titoli, la lista dei link e dei punti di riferimento, la navigazione tramite landmark e l'annuncio dei messaggi di errore nei form, sia con JavaScript attivo che disattivato (vedi Figure 4 e 5).

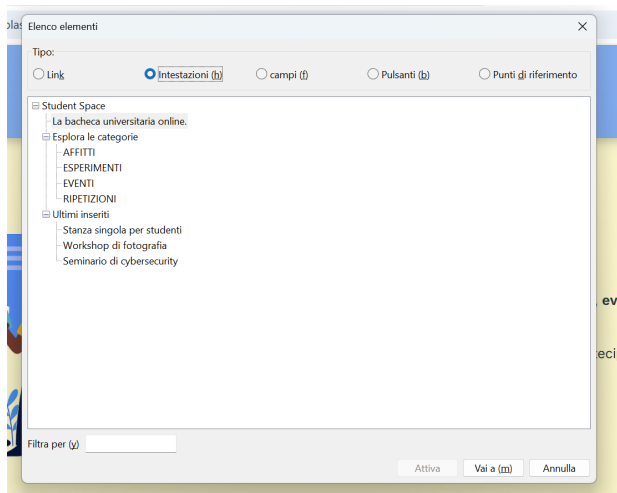


Figura 4: Gerarchia dei titoli (index.html)

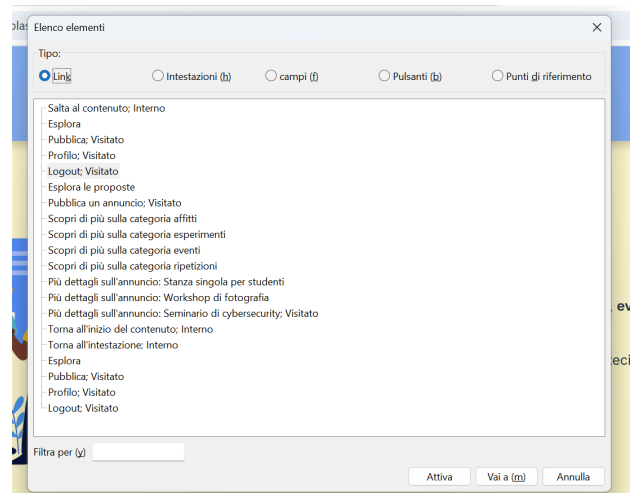


Figura 5: Lista dei link (index.html)

- **Contrast Checker** (<https://contrastchecker.com/>): per verificare il rispetto del rapporto di contrasto $\geq 4.5:1$.
- **Lighthouse, Silktide e strumenti di ispezione (F12)**: per valutazioni automatiche e test manuali su navigazione da tastiera, accessibilità dei link, dimensioni degli elementi cliccabili e altri criteri WCAG.

5.2 Tipografia e leggibilità

Per garantire una lettura chiara e accessibile sono stati scelti due font distinti:

- **Web:** *Noto Sans*, importato tramite Google Fonts.
- **Stampa:** *Times New Roman*.

La scelta del font web è stata guidata da verifiche sulla leggibilità di caratteri critici (**qp**, **db**, **00**, **iIl1**, **rn**, **m**), sull'altezza della *x* e sull'apertura delle lettere *c* e *a*. Inizialmente era stato considerato **Inter**, poi sostituito da **Noto Sans** per una migliore distinzione di glifi come **ca** e **iIl1** (vedi Figure 6 e 7).

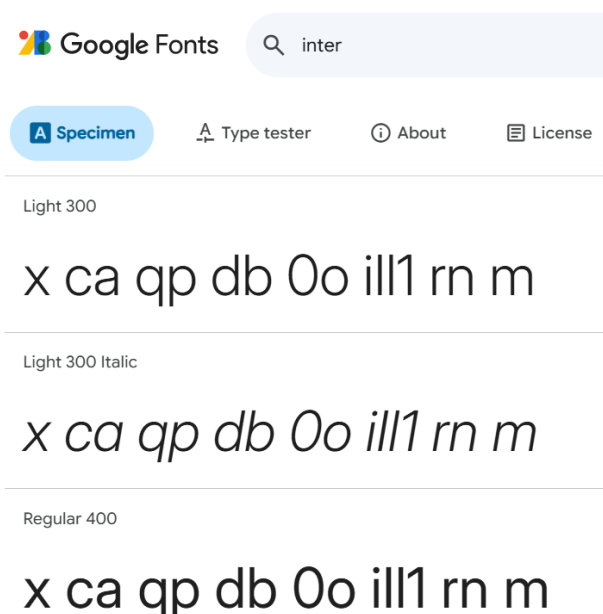


Figura 6: Font: Inter

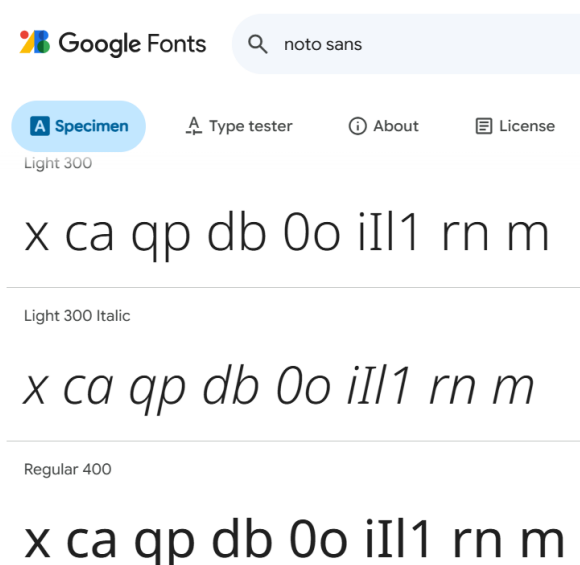


Figura 7: Font: Noto Sans

Leggibilità dei prezzi La funzione PHP `Tool::formatPrezzo()` formatta i valori numerici mostrando i decimali solo quando necessario (es. 20€ invece di 20.00€), migliorando la leggibilità.

5.3 Colori, contrasti e accessibilità mobile

Sono stati adottati diversi accorgimenti per garantire una corretta percezione dei contenuti e la riconoscibilità degli elementi interattivi:

- **Link e Call To Action (CTA):**
 - I link testuali sono sempre sottolineati.
 - I link visitati/non visitati nella navbar e nelle card annunci differiscono per colore (contrasto $2.72 \approx 3$) mentre nelle card di categoria il colore del `:visited` è associato al colore della categoria con contrasto ≥ 3 .

- I pulsanti CTA variano colore tra stato visitato e non visitato (in `background-color` e `color`).

([Apri il Sommario dei Contrasti](#) (PDF, 34 KB))

- **Accessibilità touch su mobile:**

- Gli elementi cliccabili rispettano dimensioni minime di circa $44\text{px} \times 44\text{px}$.
- Il pannello laterale dei filtri utilizza un overlay e una gestione del focus accessibile (`toggleFiltriAccessibile()`).
- I pulsanti dei caroselli hanno opacità testata e leggibile (`rgba(22, 40, 110, 0.65)`).

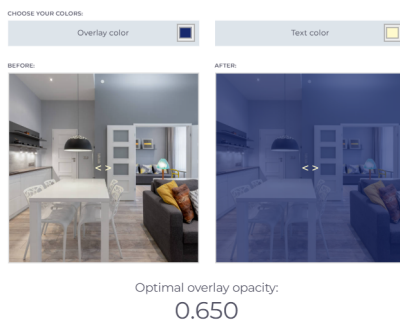


Figura 8: Opacità su immagine prova 1

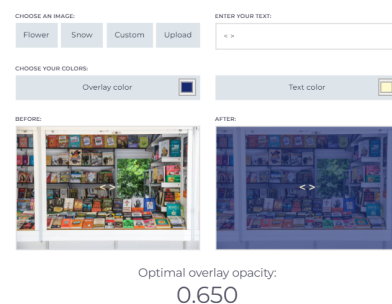


Figura 9: Opacità su immagine prova 2

6 Validazione, sicurezza e gestione degli errori

6.1 Errori lato client (JavaScript)

La validazione lato client fornisce un feedback immediato all'utente, migliorando l'usabilità e riducendo il numero di richieste non valide inviate al server. Il comportamento è attivo esclusivamente nei form che dichiarano l'attributo `data-validate`, così da evitare interferenze con form che non richiedono controlli avanzati e immediati (pre-submit).

Gestione del focus All'avvio della pagina, lo script verifica la presenza di eventuali errori generati lato server:

- in caso di errori relativi alle immagini, il focus viene portato sul primo input `file`;
- in caso di errori su campi testuali, il focus viene portato sul primo campo non valido;
- quando un errore compare per la prima volta, il contenuto del campo viene selezionato per facilitare la correzione. E' stato deciso di riportare il focus sull'input errato solo la prima volta per evitare di essere troppo restrittivi.

Eventi di validazione La validazione viene eseguita in tre momenti distinti:

- **blur**: quando un campo perde il focus, viene validato se ritenuto "validabile";

- **input:** la digitazione rimuove eventuali messaggi di errore e ripristina gli attributi ARIA;
- **submit:** tutti i campi vengono validati; se uno o più risultano non validi, l'invio viene bloccato e il focus viene portato sul primo errore.

Criteri di validazione La funzione `validateField()` applica controlli specifici in base all'ID del campo: nome, cognome, email, password, confermaPassword, città (deve corrispondere a un valore presente nella lista `datalist`), consenso-email (deve essere selezionato).

Alcuni campi vengono esclusi dalla validazione client-side: input `file`, campi relativi al testo alternativo delle immagini (`alt`), campi relativi alla marcatura decorativa (`decorativa`).

Rendering degli errori Gli errori vengono mostrati tramite:

- un contenitore `<ul>` con classi `riquadro-spieg` e `messaggi-errore-form`;
- elementi `<li>` con classe `msgErrore` e `role="alert"`;
- associazione dinamica di `aria-invalid="true"` e `aria-describedby`.

La funzione `removeError()` rimuove: il contenitore degli errori. l'attributo `aria-invalid` e eventuali riferimenti a messaggi non più presenti in `aria-describedby`.

6.2 Errori lato server (PHP)

La validazione lato server garantisce la corretta gestione degli input anche in assenza di JavaScript, assicurando coerenza, sicurezza e un comportamento uniforme in tutte le condizioni di utilizzo.

- **Tipologie di errori:** campi generici, immagini (dimensione, formato, testo alternativo), credenziali di accesso.
- **Messaggi:** generati come liste con `role="alert"`, mantenendo la stessa struttura, semantica e classi CSS dei messaggi lato client.
- **Attributi ARIA:** applicazione dinamica di `aria-invalid` e `aria-describedby` ai campi non validi, con rimozione automatica quando l'errore viene risolto.
- **Gestione del focus:** il focus viene portato automaticamente sul primo campo non valido; nel caso delle immagini, sul primo input `file`. Quando appropriato, il contenuto del campo viene anche selezionato per facilitare la correzione.
- **Errori relativi alle immagini:** controllo della dimensione (1 MB), dei formati consentiti lato server (JPG, PNG, WEBP) e dell'obbligatorietà del testo alternativo. I form accettano solo JPG per semplicità, mentre il server gestisce anche PNG e WEBP per maggiore robustezza.
- **Login:** gestione degli errori di autenticazione tramite messaggi contestuali e sistema di `redirectMessage`, che informa l'utente del motivo del reindirizzamento.

Per motivi di sicurezza, nel processo di sanificazione degli input è stata adottata la funzione `strip_tags()`, che rimuove i tag HTML e PHP dalle stringhe inserite dall'utente, prevenendo l'inserimento di codice malevolo.

La funzione elimina i tag ma non il contenuto testuale: ad esempio, da `<strong>Descrizione evento...</strong>` vengono rimossi i tag `<strong>` e `</strong>`, mantenendo il testo intero. I tag in linea e i relativi attributi vengono invece completamente eliminati.

È stata valutata la possibilità di introdurre un controllo esplicito che segnalasse all'utente la presenza di tag HTML non consentiti. Tuttavia, nel sito sono presenti campi di input per i quali non sono previsti messaggi di errore dedicati. Per mantenere coerenza nella gestione dei feedback e dei controlli, si è scelto di applicare la sanificazione a tutti i campi senza generare errori specifici.

In questo modo:

- se l'utente inserisce tag con intento semantico, il contenuto viene preservato e i tag rimossi;
- se l'input contiene un uso massiccio o sospetto di markup, viene comunque neutralizzato, assumendo che si tratti di un tentativo malevolo, considerando anche il target non tecnico della piattaforma.

6.3 Sicurezza delle password

Per garantire la sicurezza delle password degli utenti nella loro memorizzazione nel database, si è deciso di implementare un sistema di hashing delle password. In particolare, si è scelto di appoggiarsi alla funzione nativa di PHP `password_hash()`, che utilizza l'algoritmo BCrypt per l'hashing delle password.

7 Configurazione .htaccess

Il file `.htaccess` è un file di configurazione per il server web Apache, che nel nostro caso viene utilizzato per la sicurezza di alcune aree, la gestione degli errori e per rendere più leggibili gli URL.

- **Protezione aree riservate:** l'unico file ritenuto necessario da proteggere è stato il file `.env` che contiene le credenziali di accesso al database.
- **Gestione errori personalizzata:** il file `.htaccess` reindirizza gli errori 403, 404 e 500 a pagine di errore personalizzate (`403.php`, `404.php` e `500.php`).
- **URL rewriting:** il file `.htaccess` utilizza `mod_rewrite` per creare URL più leggibili e SEO-friendly, rimuovendo estensioni come `.php` dalle URL.

8 SEO e ottimizzazione per i motori di ricerca

8.1 Analisi delle possibili ricerche degli utenti

L'analisi preliminare ha individuato le principali query che uno studente universitario potrebbe digitare su un motore di ricerca. Le ricerche sono state suddivise in tre categorie:

- **Query informative generiche:** come “annunci studenti Padova”, “bacheca universitaria eventi”, “ripetizioni universitarie Padova”, “affitti studenti stanza singola”.
- **Query specifiche per categoria:** “affitti studenti vicino università”, “esperimenti retribuiti dipartimento”, “eventi universitari oggi Padova”, “ripetizioni analisi 1 Padova”.
- **Altre query:** “pubblica annuncio studenti”, “cerco coinquilini Padova”, “lezioni private matematica università”

Queste analisi hanno guidato la scelta delle *keywords*, dei *title*, delle *description* e della struttura semantica delle pagine.

8.2 Meta tag title, description e keywords

I meta tag sono stati progettati per essere coerenti con il contenuto della pagina e per rispondere alle query individuate.

Nelle pagine dinamiche (`annuncio.html`, `modificaAnnuncio.html`) i meta tag vengono generati automaticamente sulla base dei dati dell'annuncio (titolo, categoria, città).

Il titolo è prodotto tramite la funzione `Tool::titoloSEO(string, string, string, bool)`, che impone un limite di 55 caratteri, gestisce prefissi/suffissi e accorcia il testo sull'ultima parola completa.

Anche la **description** è generata dinamicamente quando necessario, così da descrivere in modo sintetico e coerente il contenuto della pagina.

Le **keywords** sono sempre presenti anche nel testo visibile (headings, **strong**), evitando discrepanze che potrebbero penalizzare il ranking (es. per Google Bombing).

8.3 Struttura semantica e accessibilità come fattori SEO

La struttura HTML è stata progettata per essere semanticamente corretta e facilmente interpretabile dai crawler:

- gerarchia coerente degli heading (`h1`, `h2`, `h3`);
- uso di landmark semantici (`header`, `main`, `nav`, `aside`, `footer`);
- testi alternativi obbligatori per le immagini;
- link descrittivi e non generici (es. “Scopri di più sulla categoria affitti”).

Questi elementi migliorano sia l'accessibilità sia la comprensione del contenuto da parte dei motori di ricerca.

8.4 Performance

Sono state adottate diverse tecniche per migliorare le prestazioni, che costituiscono un fattore di ranking:

- `loading="lazy"` per tutte le immagini non above-the-fold;
- dimensioni fisse (`width` e `height`) per prevenire layout shift;
- uso di formati ottimizzati (WebP, SVG, JPG);
- dimensione massima delle immagini inferiore a 1 MB;
- CSS modulari e caricati in modo efficiente;
- struttura del DOM pulita e priva di elementi superflui.

9 Test

Per verificare la qualità complessiva del sito, sono stati effettuati test approfonditi su più livelli: accessibilità, usabilità, robustezza del codice e comportamento multiplatforma.

9.1 Testing multiplatforma con e senza JavaScript

Il sito è stato testato su diversi browser (Chrome, Firefox, Edge) e su dispositivi differenti (desktop e smartphone). Sono state eseguite prove sia con JavaScript attivo sia con JavaScript disattivato, per verificare che i contenuti principali rimanessero accessibili e che la navigazione non risultasse compromessa. In assenza di JavaScript, sono stati controllati in particolare la leggibilità dei contenuti, la presenza di percorsi alternativi e la corretta visualizzazione delle sezioni informative.

9.2 Ordine di tabulazione e navigazione da tastiera

È stato verificato l'ordine di tabulazione da desktop e da mobile, con particolare attenzione all'*aside* dei filtri, che utilizza una tendina a scomparsa con overlay. Sono stati controllati:

- la corretta gestione del focus quando il pannello dei filtri viene aperto o chiuso;
- l'impossibilità di interagire con gli elementi oscurati dall'overlay;
- la presenza di un percorso di tabulazione lineare e prevedibile;
- la possibilità di raggiungere e attivare tutte le azioni tramite tastiera.

La navigazione da tastiera è stata testata sull'intero sito, verificando che pulsanti, link, menù e componenti interattivi fossero correttamente etichettati e attivabili.

9.3 Analisi delle prestazioni e SEO

È stato utilizzato *Lighthouse* per analizzare prestazioni, accessibilità, SEO e buone pratiche. I test hanno permesso di individuare eventuali elementi migliorabili, come ottimizzazioni delle immagini e riduzione del tempo di caricamento delle pagine.

9.4 Validazione con strumenti automatici

Sono stati utilizzati diversi strumenti per verificare la qualità del codice e l'accessibilità:

- **WAVE WebAIM**: utilizzato anche con gli stili disattivati per controllare il flusso logico dell'HTML e la corretta struttura semantica;
- **Silktide**: per un'analisi aggiuntiva di accessibilità e best practice;
- **Total Validator**: utile per validare le diverse casistiche generate dinamicamente dal PHP, verificando che ogni variante di pagina rispettasse gli standard HTML5;
- **W3B1Y**: per ulteriori controlli di accessibilità e conformità alle WCAG.

L'uso combinato di questi strumenti ha permesso di individuare e correggere errori strutturali, etichette mancanti, ruoli ARIA non necessari e possibili problemi di navigazione assistita.

9.5 Studio comportamento utenti

Per garantire che il comportamento del sito fosse chiaro, prevedibile e accessibile, abbiamo svolto una serie di test con utenti esterni al gruppo di sviluppo. Essendo noi i creatori del sistema, avremmo potuto non accorgerci di elementi poco intuitivi; per questo abbiamo coinvolto persone con profili diversi: studenti di varie facoltà, ex studenti, genitori e una persona con mobilità limitata che utilizza una sola mano. Questo ci ha permesso di osservare l'interazione da prospettive differenti e di verificare che l'esperienza d'uso rimanesse comprensibile e coerente per utenti con esigenze e competenze eterogenee.

10 Guida all'installazione del progetto

Passaggi da effettuare per un funzionamento a regime del progetto:

1. Copiare la cartella del progetto in `public_html`,
2. Creare un file `.env` nella root del progetto, con il seguente contenuto:

```
DB_HOST=localhost
DB_NAME=nome_database
DB_USER=nome_utente
DB_PASS=password_utente
```

3. Utilizzare il servizio phpMyAdmin per caricare il file `studentSpaceDB.sql` e importare la base di dati;

Viene infine fornito anche una coppia di file del dominio Docker utilizzati per lo sviluppo locale. Si noti che utilizzando il servizio di Docker si potrebbero riscontrare delle differenze per quanto riguarda il caricamento delle immagini degli annunci, in quanto tale comportamento dipende dai permessi del file system del sistema operativo ospitante.